

Rethinking NL2VIS: A Comparative Survey of LLM-Based Natural-Language-to-Visualisation Frameworks

Hock Lin TAI¹[0009–0001–0468–9672], Yunshan MA¹[0000–0003–3038–5389],
Zhaoxia WANG^{1*}[0000–0001–7674–5488]

School of Computing and Information Systems, Singapore Management University,
80 Stamford Rd, Singapore 178902, Singapore

Abstract Large Language Models (LLMs) are increasingly used to generate charts directly from natural-language instructions, yet their performance remains inconsistent across chart types and visualisation complexities. This paper examines a set of framework models that report results on comparable chart-generation benchmarks, enabling a clearer view of how these systems perform relative to one another. Through this comparative analysis, we identify persistent weaknesses in current LLM-based charting—such as unreliable library behaviours, loose alignment between code and visual output, and brittle reasoning that frequently fails in the absence of corrective mechanisms. We also highlight the broad strategies adopted across frameworks to address these shortcomings. Taken together, our findings suggest that future progress in Natural-Language-to-Visualisation (NL2VIS) will depend less on scaling models or increasing architectural complexity and more on developing lean, well-orchestrated systems with a stronger intrinsic understanding of how language, data, and visual semantics interact.

Keywords: Framework Models · LLM · NL2VIS · Visualisation

1 Introduction

Large Language Models (LLMs) are rapidly reshaping how we interact with data [1]. Yet, despite that many of these models come equipped with multi-modal and advanced reasoning capabilities, producing charts that are faithful to user intent and are suitable to the occasion remains a struggle [2–5]. It is therefore not surprising that a growing body of research has introduced specialised datasets and frameworks to strengthen specific aspects of chart generation in LLMs, with different works focusing on complementary challenges across language understanding, code generation, and visual grounding [6–10]. Yet, despite this momentum, comparing across these frameworks meaningfully remains difficult because these frameworks vary in their choice of the backbone models, and ways that these models are in turn exposed to different tailored datasets and training regimes, evaluated under different conditions and prompting strategies. Crucially, these frameworks differed in their specific agenda and areas of focus.

To bring clarity to this landscape, this paper examines a subset of framework models whose results can be compared in a consistent and interpretable manner. By placing their reported performance on a common basis and analysing their architectural strategies side by side, we highlight how different design philosophies impact accuracy, reasoning robustness, and efficiency. Our aim is to shed light not only on which approaches perform strongly, but also on the underlying reasons, and what these patterns reveal about the evolving development of Natural-Language-to-Visualisation (NL2VIS) systems. This survey argues that recent NL2VIS progress is not solely driven by scale or agent count, but increasingly by lean orchestration designs that integrate semantic planning, execution, and feedback more tightly.

This study provides four main contributions:

1. We provide a systematic comparative analysis of leading LLM-based chart-generation frameworks evaluated on comparable benchmarks.
2. We present a normalised cross-framework comparison of models evaluated on MatPlotBench [6] and Pandas-PlotBench [7], enabling fairer interpretation of published results.
3. We categorise and analyse the architectural strategies—including multi-agent pipelines, single-agent reasoning loops, and multimodal training regimes—and examine their impact on accuracy, robustness, and efficiency.
4. We propose a forward-looking perspective arguing that progress in NL2VIS depends less on model scaling and more on lean system orchestration and stronger language–data–visual semantic alignment, signalling a shift toward lightweight, semantically aligned design approaches.

2 Related Work and Analysis

The evolution of NL2VIS can be understood as a three-stage progression – from early rule-based methods to translation-based techniques and finally to LLM-based approaches [5, 11]. Early NL2VIS systems were predominantly rule-based [12–19], relying on predefined rules to make sense of natural-language queries. In turn, these were then transformed into corresponding visualisations. Subsequently, with the emergence of machine-learning translation techniques, translation-based methods [20–22] were increasingly adopted alongside rule-based approaches to translate human queries into visualisation code. More recently, LLMs have been adopted for NL2VIS because of their strong language understanding and reasoning capabilities, particularly for visualisation tasks embedded within realistic data science workflows that involve data manipulation, aggregation, and iterative chart refinement, despite studies highlighting practical challenges when applying general-purpose LLMs in such settings [23].

Given this shift toward LLM-based approaches, researchers have undertaken systematic studies to elucidate the strengths and limitations of LLM-oriented visualisation systems [24–29]. Existing benchmarks relevant to NL2VIS span a range of task formulations. While early datasets such as NL4DV [17] and Vega-Lite NL [30] corpora emphasised visualisation intent extraction or declarative

specification, DS-1000 [31] targeted general data-science code generation under naturalistic prompts; more recent benchmarks, including MatPlotBench [6] and Pandas-PlotBench [7], explicitly focus on execution-based evaluation of end-to-end NL2VIS workflows, capturing the complexity of realistic data science visualisation tasks. By evaluating rendered outputs and runtime behaviour, these benchmarks revealed failure modes that were largely undetected under earlier evaluation settings, particularly those involving semantic alignment, intent interpretation, and visual correctness.

These findings suggested that the observed failures could not be fully explained by surface-level code generation alone, pointing instead to gaps in how models connect natural-language inputs, data transformations, and visual outcomes across the visualisation pipeline. Against this backdrop, some studies explored whether making intermediate reasoning more explicit – such as through chain-of-thought prompting [5, 32] – could help bridge this gap. Among these approaches, zero-shot chain-of-thought prompting – where models are encouraged to articulate intermediate reasoning without task-specific examples – was found to provide limited benefits for recent code LLMs in NL2VIS settings, and that key visualisation errors were only visible through rendered charts rather than code alone [6]. These findings illustrated that many visualisation failures were perceptual rather than textual, exposing the limits of reasoning-only strategies.

Subsequent studies uncovered deeper structural issues with LLM-driven visualisation. Models tend to deviate from user intent due to hallucinations. These systems were frequently “struggling to faithfully adhere to user specifications” and require repeated “iterative refinement” of intermediate plots [4]. At the same time, researchers note that existing datasets did not sufficiently probe model behaviour across different libraries, instructional prompts of varying length and quality, or more demanding tabular structures [7]. Furthermore, as visualisation tasks grew more complex, it became evident that LLMs lacked mechanisms to reason across multiple modalities – natural language, data structure, and visual output – leading to systematic failures. This reflected broader dataset limitations: many corpora “lack explicit visual grounding, do not enforce runtime validation, and provide little to no supervision for recovery from failure” [8].

To navigate these limitations, researchers have looked beyond simple evaluation to explore methods that better harness the inherent language and reasoning abilities of LLMs, primarily through techniques like in-context learning [33–36], supervised fine-tuning [10, 19, 32, 37], and agentic frameworks [4, 6, 38], amongst others. While in-context learning and fine-tuning can internalise patterns derived from execution traces or visual outcomes during training or prompting, they typically operate within a monolithic inference paradigm and do not explicitly support iterative, inference-time integration of execution or visual feedback. In contrast, agentic approaches are explicitly designed to incorporate runtime execution and perceptual validation into the generation loop, making them a natural focal point for addressing the structural shortcomings identified above.

Although these agentic systems introduced specialised agents and more sophisticated coordination strategies, evaluations showed that many still struggled to synthesise multi-dimensional feedback or explore alternative solution paths, ulti-

mately falling short of producing visualisations that are “functionally precise and contextually relevant” [39]. Parallel work further revealed that many frameworks assumed access to large proprietary models, resulting in poor scalability when deployed with smaller or open-source LLMs, and often “struggled to generalize across diverse task types or model configurations,” especially under constrained capacities [40]. Underpinning these challenges was a deeper resource gap: the field lacked comprehensive, high-quality multimodal code datasets that can help develop smaller models with powerful charting capabilities that are on par or even exceeding those of larger models. However, constructing such corpora was described as a “formidable task” requiring large-scale rendering pipelines and rigorous quality control [9].

Collectively, these observations depict a field evolving through successive waves of problem discovery and solution development. The narrative moves from the early recognition of LLM limitations and inadequate benchmarks, to identifying weaknesses in purely reasoning-based methods, to the rise – and inherent constraints – of agentic systems, and ultimately to the current need for richer multimodal datasets and visually grounded learning mechanism. Each stage reflects the ongoing effort to address foundational gaps in LLM reasoning, perception, and visual grounding.

3 Benchmark Scope and Comparative Analysis Methodology

Enhancing language models’ charting capabilities is a rapid evolving area, and there are many datasets derived to strengthen specific aspects of chart generation – ranging from basic code synthesis to natural language prompt ambiguity resolution, and general multimodal chart understanding. To maintain analytical clarity in this fast-moving landscape, the scope of this paper focuses on framework models that have evaluated their performance on MatPlotBench [6] and Pandas-PlotBench [7] datasets. These two benchmarks represent a clean and methodologically consistent subset of the broader ecosystem: MatPlotBench [6] evaluates models on complex scientific visualisations that require higher-order reasoning, while Pandas-PlotBench [7] tests models on their ability to create charts from tabular data based on instructions of varying length and clarity. Both datasets have a significant proportion of chart data derived from Matplotlib Gallery [41].

Several prior works report performance on MatPlotBench [6] or Pandas-PlotBench [7] and typically compare their proposed framework against only a small set of backbone models or baseline approaches. These comparisons – such as PlotGen [4] against MatPlotAgent [6] and prompting baselines, VisCoder [8] against GPT-4o variants, or JanusCoder [9] against a limited set of coders – provide useful but modest slices of the broader landscape. In this study, we draw on the reported figures from these papers and analyse them collectively across both benchmarks. To enable clearer and more equitable comparison, our methodology normalises the reported results where necessary and consolidates performance patterns across frameworks. This provides an objective and unified

assessment that goes beyond the narrow, paper-specific evaluations commonly seen in the literature. By evaluating these two closely linked benchmark families, we hope to illustrate how alternative framework designs – whether single-agent or multi-agent – affect reasoning, code generation reliability, and overall chart correctness. In focusing on systems that calibrate their performance on these two benchmarks, other influential works including NVAgent [38], and DeepEye [15], were inevitably excluded. While these systems made several important contributions in NL2VIS, their exclusion in this study reflects methodological consistency rather than a judgement of impact. In addition, we note that while these two benchmarks are largely centred on static Matplotlib-based visualisation workflows and may not capture dynamic interactive scenarios (e.g., Plotly dashboards), streaming data contexts, or more complex real-world visualisation tasks, we hope that future studies can extend the evaluation towards more diverse benchmarks to improve generalisability and practical relevance.

The reported comparisons are based on performance statistics from original benchmark studies rather than unified experimental replication. Consequently, statistical uncertainty measures (e.g., confidence intervals) are not uniformly available, and the figures present indicative trends derived from reported point estimates.

4 Analysis and Insights from MatPlotBench

MatPlotBench [6] focuses on assessing models on their capabilities in generating complex scientific charts derived from raw data and natural-language queries. It comprises of 100 human-curated chart samples of which 75 are derived from Matplotlib Gallery [41] and remaining 25 are from OriginLab GraphGallery [42].

4.1 Frameworks Discussed

This section examines five Natural Language to Visualisation (NL2VIS) frameworks and their models – VisPath [43], PlotGen [4], DatawiseAgent [40], CoDA [39], and MatPlotAgent [6] – that share two important characteristics: (1) they have all evaluated their systems using MatPlotBench, and (2) they have all reported performance relative to MatPlotAgent, the original model established under the MatPlotBench framework. This common grounding enables meaningful cross-paper comparison despite differences in backbone models and experimental configurations.

To facilitate comparison under a unified backbone, we rescale overall scores (OS) relative to MatPlotAgent [6], which serves as the baseline (OS = 61.16). The normalisation is defined as:

$$OS_{\text{norm}} = \frac{OS}{OS_{\text{MPA}}} \times 61.16 \quad (1)$$

where OS denotes the reported Overall Score of a framework model, OS_{MPA} denotes the Overall Score of the MatPlotAgent [6] baseline under the same evaluation setting, and OS_{norm} denotes the rescaled score used for cross-framework

comparison. The constant 61.16 corresponds to the reported MatPlotAgent Overall Score on the MatPlotBench benchmark, which serves as the common reference value for normalisation. This transformation preserves proportional performance differences while anchoring all results to a common reference. The resulting normalised scores for these framework models, alongside the GPT-4 baselines, are illustrated in Figure 1 below.

4.2 Analysis of Models’ Performance

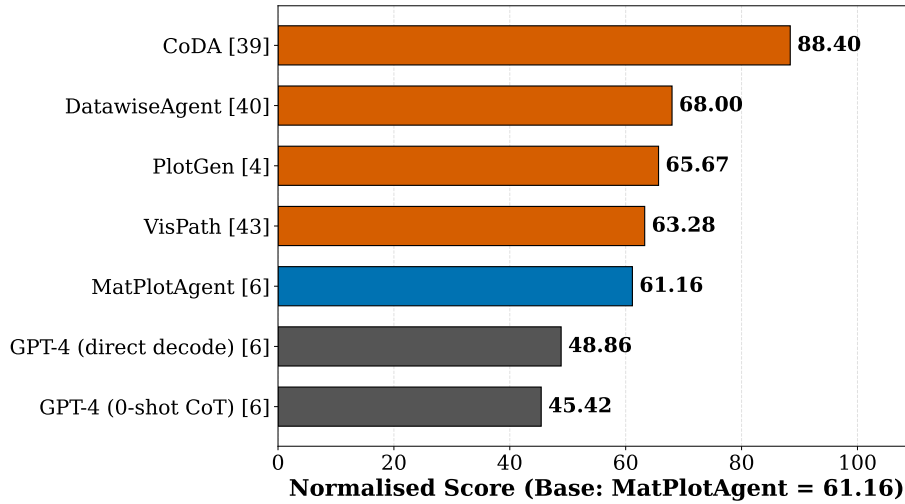


Figure 1: Reported Overall Score (OS) values are rescaled to the MatPlotAgent (MPA) baseline ($OS = 61.16$) [6] to enable cross-framework comparison. Formula used for normalisation is defined as $OS_{\text{norm}} = \frac{OS}{OS_{\text{MPA}}} \times 61.16$. Citation numbers denote original sources. Values are reproduced from reported benchmark statistics and do not include confidence interval estimates.

It is evident, from Figure 1, that all models, MatPlotAgent [6] included, exhibit stronger natural language-to-chart visualisation capabilities than GPT-4 under zero-shot Chain-of-Thought (CoT) prompting [5, 32] and GPT-4 direct decoding baselines. GPT-4, despite its strong general multimodal competence, is only able to achieve an overall score of 48.86 when evaluated under MatPlotBench [6] dataset benchmark. Notably, its performance decreases slightly to 45.42 when zero-shot CoT prompting is applied. This dip is likely due to the absence of an iterative correction process: the model generates a longer reasoning chain but lacks a self-verification or refinement loop, causing it to accumulate rather than resolve errors in chart specification or code generation.

This helps explain why the earliest multi-agent system, MatPlotAgent [6] – built on GPT-4 and equipped with a three-agent pipeline for query interpretation and code generation – achieves a much higher score of 61.16. The simple division of labour already mitigates several failure modes seen in GPT-4’s single-pass outputs.

Building on this idea, VisPath [43], another framework model that, like MatPlotAgent [6], also employs a three-agent workflow, further improves the way it parses user intent, plans the chart structure, and generates the final plotting code. It uses a multi-path agent to breakdown reasoning paths for alternative solution exploration, and this strengthens its ability to interpret ambiguous instructions and map them to MatPlotBench [6] chart specifications. PlotGen [4] and CoDA [39] further expand the agent hierarchy to five and eight agents respectively, introducing more granular roles – for query understanding, code generation, and output refinement – which collectively enable these frameworks to iteratively correct mistakes that GPT-4’s single-pass and zero-shot CoT approaches often overlook.

In contrast, DatawiseAgent [40] adopts a different strategy from the multi-agent escalation seen in VisPath [43], PlotGen [4], and CoDA [39]. Instead of increasing the number of agents, it is a single-agent system with a tightly structured workflow that controls planning, execution, and debugging. It breaks the task into small steps, runs them incrementally, and uses feedback to detect and fix errors. In addition, it has a repair and filtering process that keeps only the corrected code and maintains a clean context. This disciplined loop allows DatawiseAgent [40] to match or outperform multi-agent systems without increasing agent count. A key point to note is that, although DatawiseAgent’s [40] strongest results are still achieved with GPT-4o, a proprietary large-scale model, applying the same framework to smaller open-source backbones produces average scores that approach those of the GPT-4o configuration. This suggests that the framework’s architectural contributions – not merely the capacity of the underlying model – play a substantive role in driving its overall performance.

4.3 Key Mechanisms Of Each Framework

To facilitate our analysis of how various NL2VIS frameworks operate to render data and natural language and convert them into the required visualisations, we formulated a simple conceptual framework diagram based on a three-component schema of ‘Query Understanding’, ‘Code Generation’, and ‘Feedback’, as illustrated in Figure 2, derived from recurring architectural patterns reported in prior frameworks (e.g., MatplotAgent [6], VisPath [43], PlotGen [4]), below.

Next, we lay out in Table 1 below, how each framework corresponds to the three-component schema in Figure 2 for ease of comparison, to the best of our abilities. While not all frameworks map neatly onto these categories, this common schema offers a practical way to compare heterogeneous designs, highlight shared patterns such as iterative refinement, and surface key differences between multi-agent pipelines and single-agent systems. This standardised view enables clearer methodological contrasts across frameworks.

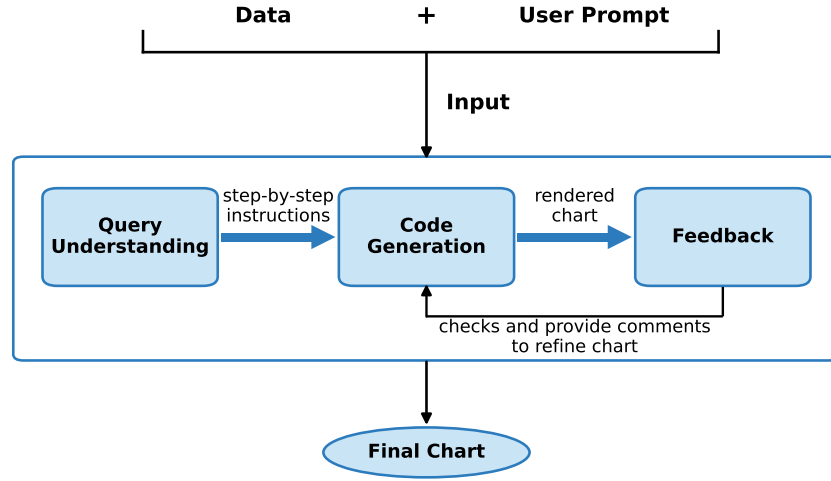


Figure 2: A conceptual three-component schema for understanding NL2VIS systems. The query understanding component interprets the natural-language user prompt and input data into step-by-step instructions; the code generation component translates these instructions into visualisation code and renders the chart; the feedback component checks the rendered chart and provides corrective comments to iteratively refine the output until a final chart is produced. The schema synthesises recurring architectural patterns observed in prior NL2VIS frameworks (e.g., MatPlotAgent [6], VisPath [43], PlotGen [4]).

Table 1: LLM-based NL2VIS frameworks evaluated on MatPlotBench [6], organised by three components — Query Understanding (how the prompt and data are interpreted into structured instructions), Code Generation (how visualisation code is produced and debugged), and Feedback (how the rendered chart is evaluated and refined). The number of agents per framework is shown in parentheses

Frameworks (# Agents)	Query Understanding	Code Generation	Feedback
MatPlotAgent[6] (3)*	Query Expansion Module rewrites queries into step-by-step instructions.	Code Agent based on instructions, generates the code to plot charts. It includes a self-debugging mechanism (cap to 3 loops) to correct errors.	Visual Agent checks and ensures: 1) generated plot aligns with data, 2) appropriate colours and labels, and 3) has structural alignment with user query.

Continued on next page

Table 1 continued from previous page

Frameworks (# Agents)	Query Understanding	Code Generation	Feedback
VisPath [43] (3)	VisPath has two agents that overlap in query understanding and code generation: Multi-Path Agent generates multiple reasoning paths based on dataset to create initial codes and corresponding 'Middle Generated Figures' for each path. Synthesis Agent integrates the best elements across the multiple reasoning paths and feedback signals to produce the final code and visualisation.		Visual Feedback Agent evaluates and refines figures.
PlotGen [4] (5)	Query Planning Agent breaks down complex queries into a sequence of executable steps using CoT prompting.	Code Generation Agent creates plots using data and streamlined steps.	Numerical Feedback Agent + Lexical Feedback Agent + Visual Feedback Agent , collectively checks and refine plots based on various elements.
DatawiseAgent [40] (1) [#]	Depth-First Search (DFS)-like Planning decompose user query into tree-structured sequence of subtasks where each branch can be pursued, revised (via backtracking), or terminated based on progress and feedback.	Incremental Execution process where it performs step-by-step code generation and execution, with each subtask implemented progressively along the active branch of the planning tree.	Self-debugging and Post-filtering are two feedback mechanisms that detect errors, repair faulty code, and refine the solution before continuing or concluding the task.
CoDA [39] (8)	Three agents - Query Analyzer , Data Processor and VizMapping Agent - interpret user query, analyse the data, determine viable visualisation goals.	Three agents - Search Agent , Design Explorer , and Code Generator - convert interpreted query and visualisation design into executable Python Code.	Two agents - Debug Agent , and Visual Evaluator - evaluate the generated code and resulting plots, identify issues, repair errors, and provide refinements.

*Query Expansion Module in MatPlotAgent [6] is considered an agent here.

[#]DatawiseAgent [40] is a single agent that does all four functions stated here.

Table 1 demonstrates how most NL2VIS frameworks can be systematically interpreted through a common three-component lens comprising query understanding, code generation, and feedback. Across both single-agent and multi-agent designs, NL2VIS systems first translate natural-language intent into structured analytical plans (query understanding), then synthesise executable visualisation code through either incremental or agent-distributed generation (code generation), and finally refine outputs via execution- or perception-driven correction mechanisms (feedback). Applying this lens uniformly across MatPlotBench [6] evaluated frameworks reveals how ostensibly diverse architectures – ranging from tightly orchestrated single-agent workflows to role-specialised multi-agent systems – instantiate the same core functional stages through different design choices. Viewed through this lens, NL2VIS systems can be understood as design instantiations that differ not in whether they perform query under-

standing, code generation, and feedback, but in how tightly and efficiently these components are orchestrated.

4.4 Analysis on Models’ Efficiency

Beyond accuracy, an important consideration in agentic NL2VIS systems is how effectively computational resources are translated into task performance. Following prior work [39], efficiency is commonly characterised by two complementary costs: token usage and LLM call counts, reflecting communication overhead and iterative coordination, respectively.

Figure 3 reports token-usage and call-count statistics for MatPlotAgent [6], CoDA [39], and VisPath [43] (with the number of agents indicated in parentheses), extracted from the CoDA paper [39], where all three frameworks are evaluated on MatplotBench [6] under a common Gemini-2.5-Pro backbone and fixed experimental conditions. Building on these reported statistics, we define two normalised efficiency indices:

$$TEI = \frac{OS}{\#Input\ Tokens + \#Output\ Tokens}, \quad CEI = \frac{OS}{\#Calls}$$

where OS denotes the reported overall score under this evaluation setting. Together, TEI and CEI measure how effectively each framework converts computational cost into task performance.

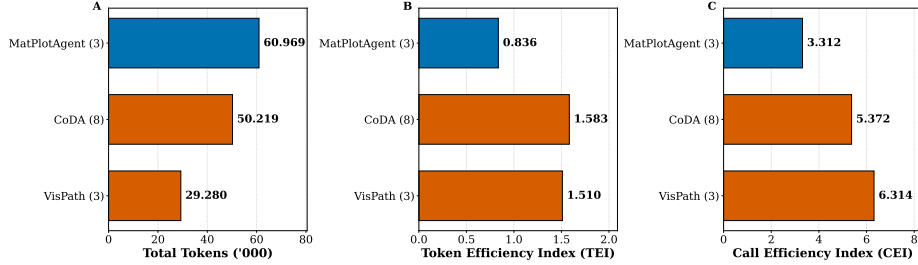


Figure 3: Token and call efficiency comparison using reported statistics from CoDA [39]. Panel A reproduces total token usage as reported. Panels B and C show derived efficiency indices defined as $TEI = \frac{OS}{\#Input\ Tokens + \#Output\ Tokens}$ and $CEI = \frac{OS}{\#Calls}$, where OS denotes the reported overall score under the common MatPlotBench [6] evaluation setting (Gemini-2.5-Pro backbone) as extracted from the original papers. Values are reproduced from reported benchmark statistics and do not include confidence interval estimates.

Interpreted in this light, TEI reflects how productively a framework transforms communication-heavy interactions (i.e., token expenditure) into task success, whereas CEI reflects how efficiently iterative coordination and refinement (i.e., model invocations) are translated into performance gains. As shown in Figure 3C, VisPath [43] achieves the highest call efficiency (CEI), consistent with its

design emphasis on lean invocation patterns and bounded traversal over alternative solution paths. In contrast, CoDA [39] attains the highest token efficiency (TEI) (Figure 3B), indicating that its larger token budget—arising from richer inter-agent communication and reflection—is effectively converted into substantially higher overall scores under the same constraints. MatPlotAgent [6], meanwhile, shows lower efficiency on both indices, indicating weaker performance relative to its token and call usage in this setting.

It is also important to recognise that the efficiency metrics presented in Figure 3 are derived from statistical results reported under a single experimental backbone, namely Gemini-2.5-Pro, as reproduced in the CoDA study. Consequently, the Token Efficiency Index (TEI) and Call Efficiency Index (CEI) should be interpreted as backbone-conditioned indicators rather than universally transferable efficiency rankings. Different backbone models may exhibit distinct reasoning behaviours, token consumption patterns, and coordination dynamics, which could alter relative efficiency outcomes across frameworks. Future studies should therefore examine cross-backbone efficiency evaluation under controlled experimental settings to obtain more generalisable insights.

Crucially, these results should not be interpreted as absolute efficiency rankings across frameworks. Rather, they highlight how distinct agentic design choices interact with a fixed execution and evaluation protocol. VisPath [43] prioritises efficient traversal with fewer model invocations, whereas CoDA [39] emphasises quality-driven, self-reflective refinement that converts higher communication overhead into improved task outcomes. Together, Figure 3A–C illustrate that efficiency in NL2VIS systems is shaped not by agent count alone, but by how computational resources are translated into performance under a shared experimental regime.

5 Analysis and Insights from Pandas-PlotBench

The Pandas-PlotBench [7] is a human-curated dataset comprising 175 visualisation tasks across three visualisation libraries: Matplotlib, Seaborn, and Plotly. It is specifically designed to evaluate how well language models can generate code from tabular data such as the Pandas DataFrame, based on instructions of varying quality.

5.1 Framework Discussed

In this section, we identified three framework models – Viscoder [8], JanusCoder [9], and AesCoder [10] – each trained on their respective proprietary datasets but evaluated on Pandas-PlotBench [7], amongst others, to evaluate their model performance. Across these frameworks, the backbone models used are open-sourced Small Language Models (SLMs), specifically Qwen variants ranging from 3B to 8B parameters.

As GPT-4o is assessed within each framework’s reported configuration, we use it as a reference baseline and compute the Relative Improvement (RI) in

average overall score (OS) as:

$$RI = \frac{OS - OS_{\text{GPT-4o}}}{OS_{\text{GPT-4o}}} \times 100 \quad (2)$$

where RI denotes Relative Improvement, OS denotes the reported average Overall Score of the framework model under the specified evaluation configuration, and $OS_{\text{GPT-4o}}$ denotes the corresponding Overall Score of the GPT-4o baseline used as the reference for comparison. The aggregated normalised metrics are depicted in Figure 4 below.

5.2 Analysis Of Models' Performance

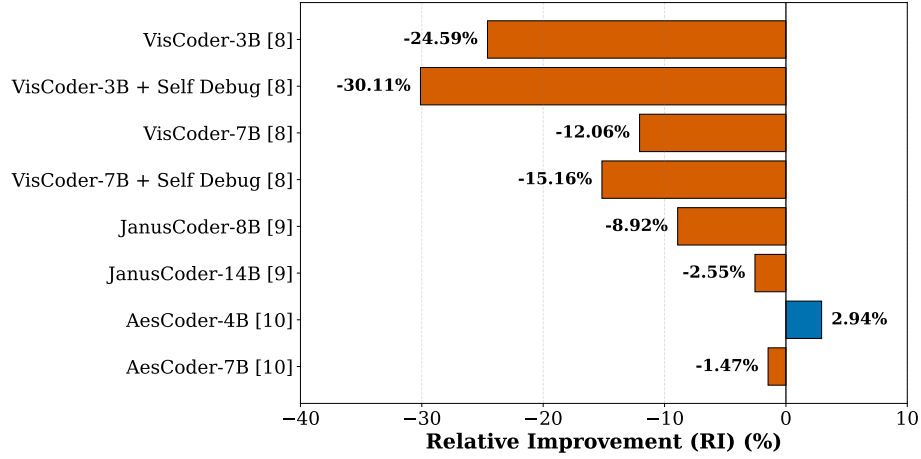


Figure 4: Relative Improvement (RI) of Viscoder [8], JanusCoder [9], and AesCoder [10] on Pandas-PlotBench [7], with their average Overall Scores (OS) normalised against the GPT-4o baseline reported in their respective settings. RI is computed as $RI = \frac{OS - OS_{\text{GPT-4o}}}{OS_{\text{GPT-4o}}} \times 100\%$. Values are reproduced from reported benchmark statistics and do not include confidence interval estimates.

VisCoder [8] – built on the domain-focused VisCode-200K [8, 44] dataset to strengthen the link between plotting code and visual output – generates reliably executable charts using a small model but, as evident from Figure 4, still lags the larger scale GPT-4o in overall visual and task fidelity. Notably, Figure 4 also highlights two open-source small models – JanusCoder [9] and AesCoder [10] – achieving performance that is close to – or even surpasses – the large, proprietary GPT-4o baseline.

The core reason JanusCoder [9] and AesCoder [10] can close the gap with GPT-4o is that their frameworks are highly specialised and are designed to address the specific weaknesses of general-purpose LLMs on plotting tasks. More specifically:

- **JanusCoder** [9] – uses a Visual-Programmatic Interface that combines visual information with code, so the model learns from both pictures and

code together after being trained on JanusCode-800K [9], a large multi-modal corpus described by the authors as containing approximately 800K text-centric and vision-centric samples. At the time of writing, only dataset descriptions and selected samples are publicly available via the official repository, while the full corpus release is pending. The strength of JanusCoder [9] stems from two complementary alignment mechanisms:

- a) Code-visual alignment: It understands the mapping between code and visual structure, and hence it knows how the code produces visual elements.
- b) Intent-semantic alignment: It understands the mapping between user intent and chart semantics, and hence it knows why a particular visual is the right choice for the instruction.

This deep alignment between – code, visual structure, user intent, and chart semantics – allows JanusCoder [9] to perform “visual reasoning” based on user prompt. Essentially, the model understands what a correct plot looks like before it even starts generating the code, thereby getting things right from the first run.

- **AesCoder** [10] – is trained with GRPO-AR, a reinforcement learning method that combines Grouped Relative Policy Optimisation (GRPO) with an agentic reward framework introduced by Xiao et al. [10]. The framework comprises three agents that jointly evaluate each generated output. The Execution Agent checks whether the code runs correctly, the Static Aesthetics Agent scores the visual layout and styling of the rendered page, and the Interactive Aesthetics Agent tests dynamic behaviours such as clicks and animations. Their reward signals are combined to guide policy optimisation. By incorporating fine-grained, domain-specific feedback from multiple expert evaluators – rather than relying on general-purpose language knowledge alone – AesCoder [10] generates outputs that are executable, visually coherent and functionally interactive. According to the reported results, AesCoder [10] matches or even surpasses GPT-4o on PandasPlotBench [7] and OpenDesign (a benchmark introduced by Xiao et al. [10]), despite being significantly smaller in scale. This suggests that task-specific, multi-agent reward systems can yield competitive performance in visually grounded code generation settings.

5.3 Key Mechanisms Of Each Framework

Table 2 below summarises the frameworks of VisCoder [8], JanusCoder [9], and AesCoder [10] by mapping their documented processes onto three functional components – Query Understanding, Code Generation, and Feedback – to enable clearer comparison across models.

Table 2: LLM-based NL2VIS frameworks evaluated on Pandas-PlotBench [7], organised by three components: query understanding (prompt and data interpretation), code generation (visualisation code production), and feedback (output validation and refinement).

Frameworks (# Agents)	Query Understanding	Code Generation	Feedback
VisCoder [8] (non-agentic)	Instruction Generation – structured NL instructions via five-component validation of query interpretation, data structure, and plot aesthetics.	Filtering and Code Block Extraction – this extracts clean visualisation code blocks that is used to train Viscoder in code generation.	Runtime Validation – executes visualisation code to validate correctness and ensure error-free rendering.
JanusCoder [9] (non-agentic)	Collection and Categorisation + Data Synthesis (Part of Curation) – performs semantic decomposition of user intent.	NL-Code coupled Data Construction (part of Curation) – where instruction-code pairs are generated and refined through guided evolution, reverse instruction, recontextualisation, and bidirectional translation.	Reward Modelling – performs quality control to ensure code correctness, visual alignment, semantic consistency, and refinement quality.
AesCoder [10] (3)	HTML Data construction + Plot data construction – that defines the instruction corpus with NL/visual/code structures for training	Model training: Supervised Fine-Tuning on AesCoder-538K + RL with GRPO-AR.	Feedback using three reward agents – Execution Agent, Static Aesthetics Agent, Interactive Aesthetics Agent – that enhance model’s abilities.

Unlike systems discussed in the MatPlotBench [6] evaluated frameworks, the Viscoder [8] and JanusCoder [9] papers do not expose internal architectural modules. The components shown in Table 2 therefore correspond to the dataset-construction pipelines (VisCode-200K [8, 44] and JanusCode-800K [9]) used to train these models rather than explicit agentic workflows. JanusCoder [9] is presented as a single multimodal model trained to support a broad spectrum of visual-programmatic tasks, with Natural-Language-to-Visualisation code generation being the only task directly comparable to NL2VIS frameworks. AesCoder [10] is similarly designed for a wider range of visual tasks, including chart plotting and WebUI editing; the processes summarised here reflect its training pipeline, which integrates supervised learning and reinforcement learning with multimodal aesthetic reward agents.

Collectively, these framework models utilise specially curated training corpora that provide rich structured multimodal signals such as correction dialogues, rendered visuals, and executable codes that are refined in the training process through specialised feedback or iterative reward mechanisms. Such dataset-centric design strategies highlight the increasing importance of data curation quality as a complementary driver of NL2VIS capability alongside architectural innovation.

6 Conclusion and Future Directions for NL2VIS

Across MatPlotBench and Pandas-PlotBench, our comparative analysis highlights both the progress and the limitations of current NL2VIS approaches. By reframing heterogeneous systems through a unified three-component schema – query understanding, code generation, and feedback – and normalising their reported performance, we provide a clearer view of how design philosophies translate into empirical outcomes. Multi-agent frameworks such as VisPath, PlotGen, and CoDA demonstrate that structured decomposition and iterative refinement can raise accuracy, yet these benefits are often accompanied by disproportionate increases in token usage and call expenses. In contrast, DatawiseAgent shows that disciplined single-agent orchestration can achieve similar or superior reasoning robustness without escalating resource demands. Meanwhile, the emergence of JanusCoder and AesCoder illustrates that small open-source models, when paired with carefully engineered multimodal corpora and specialised feedback or reward pipelines, can close the performance gap with the large-scale frontier proprietary models.

Taken together, these findings signal a significant shift in NL2VIS research. Future progress will depend on:

- Moving beyond scale as the default solution – Larger multimodal models, underpinned by the belief that "larger is better", will continue to improve performance, but scale alone is unable to resolve persistent weaknesses in reasoning and visual grounding.
- Embracing lean system design – Designing tightly orchestrated architectures, grounded in a "less is more" philosophy, that can better integrate query understanding, code generation, and feedback to achieve improved performance, instead of merely adding agents or computational resources.
- Designing for correctness from the outset, not fixing it later – Designing models that understand what the user wants and how the data should be visualised before generating code, instead of relying on repeated debugging to correct mistakes.

In short, the future of NL2VIS will depend less on scale and more on disciplined, efficient system design.

Bibliography

- [1] Adriana Alvarado Garcia, Heloisa Candello, Karla Badillo-Urquiola, and Marisol Wong-Villacres. Emerging data practices: Data work in the era of large language models. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, New York, NY, USA, 2025. Association for Computing Machinery.
- [2] Xin Wang, Zeyang Zhang, Linxin Xiao, Haibo Chen, Chendi Ge, and Wenwu Zhu. Towards multimodal graph large language model. *Science China Information Sciences*, 68(11), October 2025.
- [3] Xiao Zhang, Dongyuan Li, Liuyu Xiang, Yao Zhang, Cheng Zhong, and Zhaofeng He. Do mllms really understand the charts? *arXiv preprint arXiv:2509.04457*, 2025.
- [4] Kanika Goswami, Puneet Mathur, Ryan Rossi, and Franck Dernoncourt. Plotgen: Multi-agent llm-based scientific data visualization via multimodal feedback, 2025.
- [5] Zhihao Shuai, Boyan Li, Siyu Yan, Yuyu Luo, and Weikai Yang. Deepvis: Bridging natural language and data visualization through step-wise reasoning. *IEEE Transactions on Visualization and Computer Graphics*, 32(1):868–878, January 2026.
- [6] Zhiyu Yang, Zihan Zhou, Shuo Wang, Xin Cong, Xu Han, Yukun Yan, Zhenghao Liu, Zhixing Tan, Pengyuan Liu, Dong Yu, Zhiyuan Liu, Xiaodong Shi, and Maosong Sun. Matplotlibagent: Method and evaluation for llm-based agentic scientific data visualization, 2024.
- [7] Timur Galimzyanov, Sergey Titov, Yaroslav Golubev, and Egor Bogomolov. Drawing pandas: A benchmark for llms in generating plotting code, 2025.
- [8] Yuansheng Ni, Ping Nie, Kai Zou, Xiang Yue, and Wenhui Chen. Viscoder: Fine-tuning llms for executable python visualization code generation, 2025.
- [9] Qiushi Sun, Jingyang Gong, Yang Liu, Qiaosheng Chen, Lei Li, Kai Chen, Qipeng Guo, Ben Kao, and Fei Yuan. Januscoder: Towards a foundational visual-programmatic interface for code intelligence, 2025.
- [10] Bang Xiao, Lingjie Jiang, Shaohan Huang, Tengchao Lv, Yupan Huang, Xun Wu, Lei Cui, and Furu Wei. Code aesthetics with agentic reward feedback, 2025.
- [11] Yang Wu, Yao Wan, Hongyu Zhang, Yulei Sui, Wucui Wei, Wei Zhao, Guandong Xu, and Hai Jin. Automated data visualization from natural language via large language models: An exploratory study. *Proceedings of the ACM on Management of Data*, 2(3):1–28, May 2024.
- [12] Yiwen Sun, Jason Leigh, Andrew Johnson, and Sangyoon Lee. Articulate: a semi-automated model for translating natural language queries into meaningful visualizations. In *Proceedings of the 10th International Conference on Smart Graphics*, SG'10, page 184–195, Berlin, Heidelberg, 2010. Springer-Verlag.
- [13] Tong Gao, Mira Dontcheva, Eytan Adar, Zhicheng Liu, and Karrie G. Karahalios. Datatone: Managing ambiguity in natural language interfaces for data visualization. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*, UIST '15, page 489–500, New York, NY, USA, 2015. Association for Computing Machinery.
- [14] Vidya Setlur, Sarah E. Battersby, Melanie Tory, Rich Gossweiler, and Angel X. Chang. Eviza: A natural language interface for visual analysis. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*, UIST '16, page 365–377, New York, NY, USA, 2016. Association for Computing Machinery.

- [15] Yuyu Luo, Xuedi Qin, Nan Tang, Guoliang Li, and Xinran Wang. Deepeye: Creating good data visualizations by keyword search. In *Proceedings of the 2018 International Conference on Management of Data*, SIGMOD '18, page 1733–1736, New York, NY, USA, 2018. Association for Computing Machinery.
- [16] Bowen Yu and Claudio T. Silva. Flowsense: A natural language interface for visual data exploration within a dataflow system. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):1–11, January 2020.
- [17] Arpit Narechania, Arjun Srinivasan, and John Stasko. NL4dv: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):369–379, February 2021.
- [18] Yiru Chen and Eugene Wu. Pi2: End-to-end interactive visualization interface generation from queries. In *Proceedings of the 2022 International Conference on Management of Data*, page 1711–1725. ACM, June 2022.
- [19] Arjun Srinivasan and Vidya Setlur. BOLT: A Natural Language Interface for Dashboard Authoring. In Thomas Hoellt, Wolfgang Aigner, and Bei Wang, editors, *EuroVis 2023 - Short Papers*. The Eurographics Association, 2023.
- [20] Yuyu Luo, Nan Tang, Guoliang Li, Jiawei Tang, Chengliang Chai, and Xuedi Qin. Natural language to visualization by neural machine translation. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):217–226, January 2022.
- [21] Yuyu Luo, Nan Tang, Guoliang Li, Chengliang Chai, Wenbo Li, and Xuedi Qin. Synthesizing natural language to visualization (nl2vis) benchmarks from nl2sql benchmarks. In *Proceedings of the 2021 International Conference on Management of Data*, SIGMOD '21, page 1235–1247, New York, NY, USA, 2021. Association for Computing Machinery.
- [22] Tianqi Luo, Chuhan Huang, Leixian Shen, Boyan Li, Shuyu Shen, Wei Zeng, Nan Tang, and Yuyu Luo. nvbench 2.0: Resolving ambiguity in text-to-visualization through stepwise reasoning, 2026.
- [23] Sai Sanjna Chintakunta, Nathalia Nascimento, and Everton Guimaraes. Large language models in the data science lifecycle: A systematic mapping study, 2025.
- [24] Pere-Pau Vázquez. Are llms ready for visualization?, 2024.
- [25] Nan Chen, Yuge Zhang, Jiahang Xu, Kan Ren, and Yuqing Yang. Viseval: A benchmark for data visualization in the era of large language models, 2024.
- [26] Luca Podo, Muhammad Ishmal, and Marco Angelini. Toward a Structured Theoretical Framework for the Evaluation of Generative AI-based Visualizations. In Mennatallah El-Assady and Hans-Jörg Schulz, editors, *EuroVis Workshop on Visual Analytics (EuroVA)*. The Eurographics Association, 2024.
- [27] Jiayi Hong, Christian Seto, Arlen Fan, and Ross Maciejewski. Do llms have visualization literacy? an evaluation on modified visualizations to test generalization in data interpretation, 2025.
- [28] Saugat Pandey and Alvitta Ottley. Benchmarking visual language models on standardized visualization literacy tests, 2025.
- [29] Nam Wook Kim, Yongsu Ahn, Grace Myers, and Benjamin Bach. How good is chatgpt in giving advice on your visualization design?, 2025.
- [30] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017.
- [31] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Scott Wen tau Yih, Daniel Fried, Sida Wang, and Tao Yu. Ds-1000: A natural and reliable benchmark for data science code generation, 2022.

- [32] Luca Podo, Marco Angelini, and Paola Velardi. V-recs, a low-cost llm4vis recommender with explanations, captioning and suggestions, 2024.
- [33] Yiru Chen, Ryan Li, Austin Mac, Tianbao Xie, Tao Yu, and Eugene Wu. Nl2interface: Interactive visualization interface generation from natural language queries, 2022.
- [34] Ecem Kavaz, Anna Puig, and Inmaculada Rodríguez. Chatbot-based natural language interfaces for data visualisation: A scoping review. *Applied Sciences*, 13(12), 2023.
- [35] Paula Maddigan and Teo Susnjak. Chat2vis: Generating data visualisations via natural language using chatgpt, codex and gpt-3 large language models, 2023.
- [36] Shuaimin Li, Xuanang Chen, Yuanfeng Song, Yunze Song, and Chen Zhang. Prompt4vis: Prompting large language models with example mining and schema filtering for tabular data visualization, 2024.
- [37] Yupeng Xie, Yuyu Luo, Guoliang Li, and Nan Tang. Haichart: Human and ai paired visualization system. *Proceedings of the VLDB Endowment*, 17(11):3178–3191, July 2024.
- [38] Geliang Ouyang, Jingyao Chen, Zhihe Nie, Yi Gui, Yao Wan, Hongyu Zhang, and Dongping Chen. nvagent: Automated data visualization from natural language via collaborative agent workflow, 2025.
- [39] Zichen Chen, Jiefeng Chen, Sercan Ö. Arik, Misha Sra, Tomas Pfister, and Jinsung Yoon. Coda: Agentic systems for collaborative data visualization, 2025.
- [40] Ziming You, Yumiao Zhang, Dexuan Xu, Yiwei Lou, Yandong Yan, Wei Wang, Huaming Zhang, and Yu Huang. Datawiseagent: A notebook-centric llm agent framework for adaptive and robust data science automation, 2025.
- [41] Matplotlib Development Team. Matplotlib gallery. <https://matplotlib.org/stable/gallery/index.html>, 2025. Accessed: Dec. 01, 2025.
- [42] OriginLab Corporation. Graph gallery. <https://www.originlab.com/www/products/graphgallery.aspx>, 2025. Accessed: Dec. 01, 2025.
- [43] Wonduk Seo, Daye Kang, Hyunjin An, Taehan Kim, Soohyuk Cho, Seungyong Lee, Minhyeong Yu, Jian Park, Yi Bu, and Seunghyun Lee. Automated visualization code synthesis via multi-path reasoning and feedback-driven optimization, 2026.
- [44] TIGER Lab. Viscode-200k. <https://huggingface.co/datasets/TIGER-Lab/VisCode-200K>. Hugging Face Datasets, accessed Dec. 01, 2025.